



[Security5] - PKI

1. From the two given openssl commands, what is the difference?

• **Answer:**

- ไม่มี flag `-CAfile certificates.crt` : Verify return code: 20 (unable to get local issuer certificate)
- มี flag `-CAfile certificates.crt` : Verify return code: 0 (ok)

2. What does the error (verify error) in the first command mean? Please explain.

- **Answer:** OS = Unix & Verify return code: 20 (unable to get local issuer certificate): ไม่สามารถหาfile ที่เก็บ local certificate ที่ใช้สำหรับ verify ว่า Root CA ที่ได้มา
→ เราไม่อาจ Authentication ได้ว่า `chula.ac.th` คือ chula จริงๆ

3. Copy the server certificate (beginning with -----BEGIN CERTIFICATE----- and ending with -----END CERTIFICATE-----)

and store it as `twitter_com.cert`. Use the command `openssl x509 -in twitter_com.cert -text` to show a text representation of the certificate content. Briefly explain what is stored in an X.509 certificate (i.e. data in each field).

- **Answer:** สามารถแบ่งได้เป็น 4 ส่วนหลักๆ

1. *Certificate Data:*

- Version(X.509), Serial Number, Algorithm ที่ใช้, CA ที่ออกให้, Validity(ระยะเวลาที่ Certificate ใช้ได้), ผู้ถือครอง(Subject), Info public key(Algorithm ที่ใช้, Size, ...)

2. *X.509v3 Extension:*

- Public key CA, รหัส Hash, Key usage, CA:False(อันนี้ไม่ใช่ CA), Certificate Policies, AIA, SAN, SCT

3. Signature of CA ที่ออก Certificate นี้

4. PEM Encoder (Encoder ของ Certificate ที่เอามาอ่าน)

4. From the information in exercise 3, is there an intermediate certificate? If yes, what purpose does it serve?

Hint: Look for an issuer and download the intermediate certificate. You may use the command `openssl x509 -inform der -in intermediate.cert -text` to show the details of the intermediate certificate. (Note that the `-inform der` is for reading the DER file. The default file format for x509 is the PEM file.)

• **Answer:**

- Is there an intermediate certificate? **Yes** เพราะ
 - `intermediate.cert` มี `CA: TRUE` แปลว่ามีต่อ CA ที่มารับต่ออีก
 - หรือ จาก `openssl s_client ...` พบว่ามีเรียก CA ต่อกัน 3 ชั้น
- purpose of intermediate CA:
 1. ช่วยให้ไม่มีการเรียกใช้ root CA ไม่มีการใช้งานบ่อย → ช่วยให้ Root CA สามารถลดโอกาสที่ใช้บ่อยแล้ว private key รั่วได้
 2. Case intermediate CA private key รั่วก็ยังไม่ Root CA คอย revoke ได้

3. สามารถแยกย่อย intermediate CA ให้ออก Certificateตามหน้าที่ต้องการได้
5. Is there an intermediate CA, i.e. is there more than one organization involved in the certification? Say why you think so.
- **Answer:**
 - มี intermediate CA คือ Sectigo Public Server Authentication CA DV R36
 - is there more than one organization involved? **No** มีแค่ Sectigo Limited เพราะ Chain of trust มี intermediate CA & Root CA มี `O=Sectigo Limited` เหมือนกัน
6. What is the role of ca-certificates.crt?
- **Answer:** Bundle ของ Certificate ของ Root CA เพื่อที่จะไม่ต้องเก็บ Certificate ที่เชื่อได้เยอะๆ fileนี้ก็จะเก็บแค่ Certificate ของ Root CA จะได้ไม่ต้องเก็บ Certificate เยอะ
7. Explore the ca-certificates.crt. How many certificates are in there? Give the command/method you have used to count.
- **Answer:** มี 121 Certificate
 - Solution: Count `-----BEGIN CERTIFICATE-----` เพราะ PEM format จะขึ้นต้นอย่างงั้นเสมอ
 - `(Select-String -Path .\certificates.crt -Pattern "-----BEGIN CERTIFICATE-----").Count`
8. Extract a root certificate from ca-certificates.crt. Use the openssl command to explore the details. Do you see any Issuer information? Please compare it to the details of twitter's certificate and the details of the intermediate certificate.
- **Answer:** กำหนดให้ `root.crt` คือ certificate ที่ดึงมาจาก `ca-certificates.crt`
 - `root.crt` มีความต่างจาก certificate ของ `chula.ac.th` & `intermediate CA` คือ
 1. ค่าต่างๆใน Issuer = Subject (ค่าของคนออก = คนที่ได้รับ) คือได้รับการ Verify ด้วยตัวเอง
 - **Subject:** `CN=ACCVRAIZ1, OU=PKIACCV, O=ACCV, C=ES`
 - **Issuer :** `CN=ACCVRAIZ1, OU=PKIACCV, O=ACCV, C=ES` (รวมถึงในส่วน Extensions ค่าของ `Subject Key Identifier` และ `Authority Key Identifier` ก็เป็นค่าเดียวกัน คือ `D2:87:B4:E3:...`)
 2. `CA:TRUE` : root.crt & intermediate CA, `CA:FALSE` : chula.ac.th
9. If the intermediate certificate is not in a PEM format (text readable), use the command to convert a DER file (.crt .cer .der) to PEM file.
`openssl x509 -inform der -in certificate.cer -out certificate.pem.` (You need the pem file for exercise 10.)
- **Answer:** ☒ Change to PEM file
10. From the given python code, implement the certificate validation. Use your program to verify the certificates of:
 Twitter, google, www.chula.ac.th, classdeedee.cloud.cp.eng.chula.ac.th
- **Answer:**

```
45 if __name__ == '__main__':
46     custom_links = [
47         'https://twitter.com',
48         'https://www.google.com',
49         'https://www.chula.ac.th',
50         'https://classdeedee.cloud.cp.eng.chula.ac.th'
51     ]
52     verify(custom_links)
53
```

Problems Output Debug Console Terminal Ports powershell + - @ [] ... [] x

```
PS D:\University_year3\Computer_Security\Security4_PKI> python .\main.py
[*] Checking: twitter.com
--> Certificate verified

[*] Checking: google.com
--> Certificate verified

[*] Checking: www.chula.ac.th
--> Certificate verified

[*] Checking: classdeedee.cloud.cp.eng.chula.ac.th
--> Certificate verified

PS D:\University_year3\Computer_Security\Security4_PKI>
```

```
from OpenSSL import crypto
import pem

def verify():
    with open('./target.pem', 'r') as cert_file:
        cert = cert_file.read()

    with open('./intermediate.pem', 'r') as int_cert_file:
        int_cert = int_cert_file.read()

    pems = pem.parse_file('./certificates.crt')
    trusted_certs = []
    for mypem in pems:
        trusted_certs.append(str(mypem))

    trusted_certs.append(int_cert)

    verified = verify_chain_of_trust(cert, trusted_certs)

    if verified:
        print('Certificate verified')
    else:
        print('Certificate not verified')

def verify_chain_of_trust(cert_pem, trusted_cert_pems):
    certificate = crypto.load_certificate(crypto.FILETYPE_PEM, cert_pem)
    # Create and fill a X509Store with trusted certs
    store = crypto.X509Store()
    for trusted_cert_pem in trusted_cert_pems:
        trusted_cert = crypto.load_certificate(crypto.FILETYPE_PEM, trusted_c
ert_pem)
        store.add_cert(trusted_cert)
```

```

# Create a X509StoreContext with the cert and trusted certs
# and verify the the chain of trust

store_ctx = crypto.X509StoreContext(store, certificate)

# Returns None if certificate can be validated
result = store_ctx.verify_certificate()
if result is None:
    return True
else:
    return False

if __name__ == '__main__':
    custom_links = [
        'https://twitter.com',
        'https://www.google.com',
        'https://www.chula.ac.th',
        'https://classdeedee.cloud.cp.eng.chula.ac.th'
    ]
    verify(custom_links)

```

11. Nowadays, there are root certificates for class 1 and class 3. What uses would a class 1 signed certificate have that a class 3 doesn't, and vice versa?

• **Answer:**

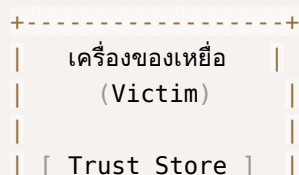
- สิ่งที Class 1 ทำได้ แต่ Class 3 ไม่ตอบโจทย์หรือไม่เหมาะสม:
 - Certificate ที่ต้องการไม่เปิดเผยตัวตนเช่นผ่าน emailว่าเป็นของจริง แต่ class 3 ต้องตรวจยืนยันตัวตน
 - Certificate ที่ไม่ต้องการความรวดเร็ว และค่าใช้จ่ายต่ำกว่า
- สิ่งที Class 3 ทำได้ แต่ Class 1 ไม่สามารถทำได้
 - OS บางversion อาจจะต้องการ classชั้นต่ำคือ 3 เพื่อการ Run
 - Online bank ที่ต้องการความ Secure เลยอาจจะต้องการ Classที่สูง
 - non-repudiation: การลงนามเอกสารที่มีผลผูกพันทางกฎหมาย เพื่อที่จะสามารถยืนยันตัวตนได้
 - การคุ้มครองความเสียหายทางการเงิน (CA warranty)

12. Assuming that a Root CA in your root store is hacked and under the control of an attacker, and this is not noticed by anyone for months.

a. What further attacks can the attacker stage? Draw a possible attack setup.

• **Answer:**

- *man-in-the-middle attack*: เมื่อ user มีการร้องขอ bank.com → คนร้ายเปลี่ยนเส้นทาง trafficมาที่คนร้าย & ส่ง Certificateที่สร้างขึ้น → user เชื้ออาจส่งข้อมูลที่อันตรายให้ เช่น password
- *supply chain ผ่าน code signing*: การให้ Certificate แก่ malware/ softwareที่อันตราย





b. In the attack you have described above, can we rely on CRLs or OCSP for protection? Please explain

- **Answer:** CRLs or OCSP ไม่สามารถ protection ได้เพราะ CRLs or OCSP ถูกควบคุมด้วย Root CA ซึ่ง Root CA ถูก hack ไปแล้ว
 → จึงเหมือนให้ Root CA ออกใบว่าเรา hack อะไรไปบ้าง
 → ทางที่ทำได้คือเมื่อรู้ว่า Root CA โดน hack ก็ทำการถอดถอน Root CA ออกจาก trust store ของ OS/ Browser